

KV260 Üzerinde 3D GPU

PS+PL Ortak Tasarım · Proje Tasarım Raporu

AMD Kria KV260 (Zynq UltraScale+ MPSoC) üzerinde, sıfırdan RTL ile inşa edilen; ARM/Ubuntu host'un sürdüğü, PL fabric'te koşan, 4 GB DDR4'ü VRAM olarak kullanan ve HDMI'a çıkan gerçek-zamanlı, perspektif-doğru dokulu ve ışıklı bir 3D rasterization GPU'su. Öğrenme odaklı tasarım belgesi.

Tasarım raporu · Sürüm 1.0 · Temmuz 2026

1 Özet

Bu proje, KV260 üzerinde bir 3D rasterization GPU'sunu **PS+PL ortak tasarımıyla** sıfırdan kurar. ARM Cortex-A53 (Ubuntu) oyun mantığını, input'u ve kamera/dönüşüm matrislerini yürütür; PL fabric'teki rasterizer üçgenleri piksele çevirir; 4 GB DDR4 vertex/doku/framebuffer/z-buffer'ı tutar; sonuç HDMI'dan çıkar. Nihai hedef gezilebilir, dokulu ve ışıklı bir 3D sahne / mini oyun. Render detayı bilinçli olarak en ileri seviyede seçildi: **perspektif-doğru dokulama + gölgeleme**.

Platform	AMD Kria KV260 (Zynq UltraScale+ MPSoC, K26 SOM)
Host (PS)	Dört çekirdekli ARM Cortex-A53, Certified Ubuntu for Kria
Hızlandırıcı (PL)	Sıfırdan RTL (Verilog/SystemVerilog) rasterizer
VRAM	4 GB DDR4 — framebuffer, z-buffer, doku, geometri
Video çıkışı	HDMI 1.4, platform video pipeline üzerinden
Hedef çözünürlük	1280×720 (720p); bring-up 640×480
Render	Perspektif-doğru dokulu + Gouraud ışık, z-buffer, double-buffer
Aritmetik	Fixed-point Q16.16
FPS hedefi	≥30 fps (60 fps stretch)
Mimari	PS+PL, AXI (HP: veri yolu, Lite: kontrol)
Amaç	Öğrenme

2 Amaç ve öğrenme hedefleri

Bu bir üretim ürünü değil, bir öğrenme projesi; rapor da buna göre "neden"leri açıklar, sadece "ne"yi değil. Proje tamamlandığında kazanılacaklar:

- Uçtan uca 3D grafik pipeline'ı — yazılımda değil, donanımda.
- Gerçek CPU↔GPU sistem tasarımı: komut gönderimi, paylaşımlı VRAM, senkronizasyon, page-flip — yani gerçek GPU'ların host ile arayüzü.

- MPSoC üzerinde AXI, DMA, bellek bant genişliği ve önbellekleme.
- Fixed-point datapath tasarımı, pipelining, bellek arbitrajı.
- Zynq UltraScale+ PS+PL akışı; Vivado/Vitis/Ubuntu; DRM/KMS ile HDMI çıkışı.

3 Hedef platform: Kria KV260

- **Zynq UltraScale+ MPSoC (K26 SOM):** PS'te dört çekirdekli ARM Cortex-A53 (+ çift Cortex-R5F), PL'de programlanabilir fabric.
- **4 GB 64-bit DDR4** (PS'e bağlı), 16 GB eMMC, ~26.6 Mb on-chip SRAM (BRAM/URAM).
- **Video:** HDMI 1.4 ve DisplayPort 1.2a (aynı anda biri aktif).
- **Yazılım:** Vivado, Vitis, PetaLinux / Certified Ubuntu for Kria.

VRAM erişim yolu

DDR bellek denetleyicisi PS'in içinde. PL, 4 GB DDR4'e PS interconnect üzerinden AXI HP (high-performance) portlarıyla ulaşır. Framebuffer, z-buffer ve dokular burada yaşar; hem PL hem ARM erişebilir.

4 Sistem mimarisi (PS+PL)

İş bölümü, gerçek bir CPU+GPU sistemine birebir oturuyor: ARM host, PL ise hızlandırıcı GPU.

Blok	Sorumluluk
PS — ARM/Ubuntu	Oyun döngüsü, input (Linux evdev), kamera & nesne dönüşümleri (MVP), sahne yönetimi, nesne-seviyesi culling, frame başına komut/geometri buffer'ını DDR4'te oluşturma, video pipeline (DRM/KMS) konfigürasyonu, PL'yi başlatma ve IRQ ile senkron.
PL — fabric	DDR4'ten komut/vertex okuma (AXI master), rasterization + doku + z-test + gölgeleme, framebuffer'ı DDR4'e yazma; kontrol/status için AXI-Lite slave.
DDR4 — VRAM	Vertex/index/komut buffer'ları, doku atlası, framebuffer A/B (double-buffer), z-buffer.
Video pipeline	Framebuffer DDR4 → frmbuf/VDMA → Video Mixer → HDMI TX → ekran.

Frame akışı (ping-pong): PS, frame N+1'in komut buffer'ını kurarken PL frame N'i render eder → PL bitince IRQ → PS double-buffer page-flip yapar → video pipeline yeni buffer'ı tarar. İki taraf sürekli örtüşerek çalışır.

Tasarım kararı — vertex transform nerede koşar?

Başlangıçta PS (matris matematiği ARM'da; PL'e ekran-uzayı üçgenleri gider) — böylece piksele hızlı varılır ve asıl donanım işi olan rasterizer öne çıkar. Geometry aşaması (MVP + perspective divide + clip + cull) daha sonra PL'e taşınır (Faz 7) — bu da "tam GPU" adımı. Hedef mimari tam-PL geometry, ama inşa PS-transform ile başlar.

5 Render pipeline (nihai hedef)

- Geometry:** MVP → clip space; near-plane clipping; perspective divide ($1/w$) → NDC; viewport → ekran; backface culling. (Q16.16; 4×4 matris = 16 MAC → DSP slice.)
- Triangle setup:** edge function katsayıları, bounding box; perspektif-doğru için $1/w$ ve öznitelik/w kurulumu.

- 3 **Rasterization:** edge-function ile piksel kapsama (bbox taraması, inkremental adder'lar), barycentric ağırlıklı fragment akışı.
- 4 **Perspektif-doğru interpolasyon:** u/w , v/w , $1/w$ (ve renk/normal) interpolate edilir; piksel başına $1/w$ 'ye bölünerek gerçek u,v geri elde edilir.
- 5 **Doku örnekleme:** DDR4'teki dokudan texel (BRAM texture cache + miss'te AXI fetch); nearest → (stretch) bilinear.
- 6 **Gölgeleme:** Gouraud diffuse ışık (köşe interpolasyonlu), doku ile modüle. (Stretch: per-pixel.)
- 7 **Depth test:** z-buffer oku-karşılaştır-yaz (depth/tile cache ile DDR trafiği azaltılır).
- 8 **Framebuffer yaz:** renk DDR4'e burst-write.

```
// her frame (PS + PL birlikte)
PS: input & kamera -> M,V,P matrisleri -> komut buffer (DDR4)
PS: PL'yi tetikle (CTRL.start), IRQ bekle
PL: for each triangle:
    setup (edge fn, bbox, 1/w, attr/w)
    for each pixel in bbox:
        coverage? -> barycentric
        persp-correct interp -> u,v,z, isik
        texel = tex_cache[u,v]; color = shade(texel, isik)
        if z < zbuf: write color + z
PL: done -> IRQ; PS: page-flip (double buffer)
```

PS host'u frame'i hazırlar ve tetikler; PL rasterizer datapath'i piksel üretir. Ping-pong ile örtüşür.

6 PL rasterizer mikromimarisi

Modüller (stage'ler arası valid/ready handshake + FIFO'lar; datapath pipelined ve stall edilebilir):

- **Komut/vertex fetch** — AXI master (HP), DDR4'ten komut ve geometri okur.
- **Geometry unit** (Faz 7) — 4×4 MAC dizisi (DSP), perspective divide (pipelined reciprocal / Newton-Raphson), viewport.
- **Clip unit** — near-plane clipping.
- **Triangle setup** — edge katsayıları, bbox, perspektif-doğru kurulum.
- **Rasterizer core** — bbox iterator + edge evaluator'lar (inkremental), fragment akışı.
- **Interpolator** — barycentric interpolasyon; piksel başına reciprocal.
- **Texture unit** — adres üretimi + BRAM/URAM texture cache + miss'te AXI fetch + (bilinear) filtre.
- **Shader/blend** — ışık combine, doku modülasyonu.
- **Depth unit** — z-buffer read-modify-write + depth cache.
- **Framebuffer writer** — AXI master, burst-write.
- **Kontrol/status registerları** — AXI-Lite slave.
- **Tepe kontrol FSM** — frame'i sıralar, ağır işi datapath'e havale eder.

```
IDLE -> FETCH_CMD
-> FOR_EACH_TRI { SETUP -> RASTER -> SHADE -> ZTEST_WRITE }
-> FRAME_DONE (IRQ) -> IDLE
```

Tepe kontrol FSM'i: saf RTL FSM + datapath — mimari A'da da rasterizer'ın donanım saflığı korunur.

7 PS yazılımı (Ubuntu)

ARM üzerinde C/C++ bir uygulama. **Oyun döngüsü:** input oku (Linux evdev — klavye/gamepad) → kamera ve nesne matrislerini güncelle → sahne graph + nesne-seviyesi culling → komut buffer'ını DDR4'te oluştur → PL'yi tetikle (AXI-Lite) → IRQ bekle → DRM page-flip. Ayrıca aynı pipeline'ın bir **C golden model**'i tutulur: hem Faz 1'in yazılım renderer'ı, hem de donanımı satır satır doğrulamak için referans.

8 Bellek sistemi

DDR4 yerleşimi (720p, yaklaşık boyutlar):

Bölge	Boyut (720p)
Framebuffer A/B (RGB888)	$1280 \times 720 \times 3 \approx 2.64 \text{ MB} \times 2 \approx 5.3 \text{ MB}$
Z-buffer (16-bit)	$1280 \times 720 \times 2 \approx 1.76 \text{ MB}$
Doku atlası	içeriğe göre birkaç MB
Vertex / index / komut buffer	KB-MB mertebesi
Toplam	4 GB içinde ihmal edilebilir

AXI: PL master'ları HP portlarıyla DDR4'e, AXI-Lite GP portuyla kontrol. **On-chip (~3.3 MB):** texture cache, depth/tile cache, FIFO'lar. **Bant genişliği:** 720p60 scanout okuması $\approx 166 \text{ MB/s}$ (ihmal); asıl trafik rasterizer okuma/yazma + doku fetch (overdraw ve cache hit oranına bağlı). DDR4 birkaç GB/s mertebesinde olduğundan iyi bir cache tasarımıyla bant genişliği rahat. **Tearing:** double-buffer + vblank ile senkron page-flip.

9 Video çıkış yolu

Framebuffer DDR4 → frmbuf/VDMA read → (Video Mixer) → HDMI TX Subsystem → HDMI konektör. 720p60 için pixel clock 74.25 MHz. Ubuntu'da bu, DRM/KMS + platform video pipeline ile yürür: PL rasterizer bir DRM framebuffer'ına yazar, uygulama page-flip yapar. **Baseline (önerilen):** platform pipeline'ını kullan, HDMI TX'i yeniden yazma. **İleri seçenek:** PL'de custom HDMI TX (TMDS encoding + yüksek hızlı serialization).

10 Arayüz ve komut gönderimi

Kontrol arayüzü — AXI-Lite register haritası (taslak):

Register	Offset	İşlev
CTRL	0x00	start biti (render başlat)
STATUS	0x04	busy / done
FB_BASE_A / _B	0x08 / 0x0C	framebuffer taban adresleri (double buffer)
ZB_BASE	0x10	z-buffer taban adresi
CMD_BASE / COUNT	0x14 / 0x18	komut buffer adresi ve komut sayısı
RES_X / RES_Y	0x1C / 0x20	çözünürlük
MODE / IRQ	0x24 / 0x28	mod bayrakları / kesme durumu

Komut buffer formatı: DDR4'te bir draw komutları listesi — set_texture(base, dims), set_matrix (PL geometry aşamasında), draw_triangles(vertex_base, count); ya da daha basit olarak öznitelikli üçgen dizisi. **Senkron:** PS frame N+1'i kurarken PL frame N'i render eder; PL bitince IRQ; PS page-flip. Ping-pong.

11 Tasarım kararları ve gerekçeleri

Karar	Seçim	Gerekçe
Aritmetik	Fixed-point Q16.16	FPGA float sevmez; Q16.16 3D için yeterli hassasiyet, geniş intermediate'lerle edge-function taşması önlenir
Çözünürlük	720p hedef, 480p bring-up	1080p = 2.25× piksel/fill/bandwidth; 720p tatlı nokta, HDMI'da modern görünüm; 480p erken getirme
Renk / z	RGB888 / 16-bit z	HDMI için doğal; 16-bit z tipik sahne için yeterli
İnterpolasyon	Perspektif-doğru	Dokulu olduğu için zorunlu; affine dokuyu bozar (piksel başı reciprocal)
Culling	Backface (winding)	Fill'i yaklaşık yarıya indirir
Clipping	Near-plane	Kamera arkası ve ~0'a bölme artefaktlarını önler
FPS	≥30 (60 stretch)	Gezilebilir his için yeterli; erken hedef gerçekçi
Doku filtre	Nearest → bilinear (stretch)	Önce doğruluk, sonra kalite
Işık	Gouraud diffuse → per-pixel (stretch)	İleri ama uygulanabilir baseline

12 Geliştirme planı ve milestone'lar

Faz	Ne	Çıktı
0	Platform bring-up: Ubuntu + HDMI (DRM/KMS test pattern), AXI hello-world, sim (Verilator/cocotb)	C'den ekrana bir framebuffer; PL AXI-Lite peripheral'ı ARM'dan poke
1	Yazılım referans renderer: tüm dokulu-ışıklı rasterizer C'de → DRM framebuffer → HDMI	Ekranda yazılım gezilebilir sahne (golden model)
2	PL framebuffer writer + scanout: PL DDR4'e pattern yazar, DRM tarar	PL→DDR4→HDMI yolu kanıtlandı
3	PL 2D üçgen rasterizer: edge-function core, flat renk, PS ekran-uzayı üçgeni besler	Donanımda ilk üçgen
4	Perspektif-doğru interpolasyon + texture unit + texture cache	Donanımda dokulu üçgen
5	Z-buffer + depth cache	Üst üste binen geometri doğru çözülür
6	Işık (Gouraud) + fragment shading	Tam fragment aşaması: ışıklı dokulu
7	Geometry aşamasını PL'e taşı (MVP + persp divide + clip + cull)	PS sadece model-uzayı + matris verir; tam GPU

8	ARM oyun döngüsü + double-buffer + optimizasyon (bandwidth/cache/pipeline)	Gezilebilir mini oyun, hedef FPS
---	--	----------------------------------

13 Araç zinciri ve iş akışı

- **Vivado** — PL RTL sentez/implement + block design (AXI ve video IP entegrasyonu).
- **Vitis** — gömülü yazılım/hızlandırma gerekirse.
- **Certified Ubuntu for Kria** — işletim sistemi ve sürücüler; PL tasarımı Kria'da bir "accelerated application" olarak yüklenir (fpga-manager / xmutil + device-tree overlay ki Linux sürücüler PL IP'ye bağlansın).
- **Simülasyon** — Verilator + cocotb ile rasterizer, C golden model'e karşı çapraz doğrulama.
- **HDMI doğrulama** — DRM/KMS + modetest.
- **Sürüm kontrolü** — Git.

14 Riskler ve azaltma

Risk	Azaltma
DDR bant genişliği / fill (720p + overdraw + doku)	Texture cache + depth tile cache + backface cull; 480p'den başla, sonra 720p'ye ölçekle
AXI entegrasyon karmaşıklığı	AXI-Lite hello-world + basit DMA'dan başla; Xilinx AXI infrastructure IP kullan
Kria/Ubuntu HDMI bring-up cilveli olabilir	Rasterizer'dan önce platform HDMI'ı (test pattern) çalıştır (Faz 0)
Fixed-point taşma (edge functions)	Intermediate'leri genişlet, aralık analizi yap, C golden model'e karşı test et
Scope creep (gezilebilir oyun)	Fazlı milestone'lar; her faz bağımsız demolanabilir
Kria firmware / device-tree overlay öğrenme eğrisi	AMD Kria KV260 dokümanları ve örneklerini takip et

15 Kaynaklar

- **Yazılım rasterizer / matematik:** tinyrenderer (ssloy), javidx9 3D Graphics Engine, Scratchapixel (perspektif-doğru interpolasyon).
- **FPGA grafik:** Project F (projectf.io) — framebuffer, çizgi/üçgen, animasyon, fixed-point, division, video timing.
- **KV260 / Kria:** AMD Kria KV260 dokümanları, Vitis platform, Certified Ubuntu for Kria, KV260 smartcam / video-pipeline örnekleri (frmbuf + Video Mixer + HDMI referansı).
- **Zynq US+ IP:** Xilinx VDMA/frmbuf, Video Mixer, HDMI TX Subsystem ve ilgili PG dokümanları.